

به نام خدا

گزارش کارهای انجام شده در آزمایشگاه
برای آزمایش 5 – درس آزمایشگاه
طراحی سیستم های دیجیتال

امیر محمد شربتی (402106112) – پوریا رحمانی (402111418)

1404/9/17

در این آزمایش باید یک ضرب کننده booth با قابلیت های بیشتر پیاده سازی می‌کردیم. ضرب کننده ای که قابلیت چند بیت شیف در یک کلاک را داشته باشد. همچنین در این طراحی می‌بایست datapath و control unit را جدا می‌کردیم.

تمام این موارد پیاده سازی شدند و در پیش گزارش به صورت کامل ارسال شدند.

قبل از توضیح در مورد کار های انجام شده در آزمایشگاه، باید نکته ای را عرض کنم. همانطور که با جنابعالی صحبت کردیم و در پیش گزارش هم مفصل نوشتیم، متأسفانه کد اولیه را نتوانستیم تا قبل از دوشنبه هفته گذشته (یعنی تا پایان روز 9 آذر) به صورت کامل اصلاح کنیم. (ما سعی کردیم بهترین حالت پیاده سازی را انجام دهیم. یعنی پیاده سازی برای ضرب اعداد با تعداد بیت متغیر.) اما بعدش کلی (واقعا خیلی، حدود یک روز) وقت گذاشتیم تا کد را دیباگ کنیم. در نهایت پس از کلی تلاش، موفق شدیم یک کد درست و بدون ایراد پیاده سازی کنیم.

این را با فرض اینکه با فایل گزارش آپلود خواهیم کرد (یا با همین فایل) نوشتیم. اما بعدش متوجه شدیم می‌شود آن را آپلود کرد. بنابراین یکی از ما فایل های اصلاحی و پیش گزارش کامل آزمایش 5 را هم در کنار فایلی که یکشنبه (9 آذر) آپلود کردیم، چند روز بعد (13 آذر) آپلود کردیم و نفر دیگر آپلود نکرد تا این فایل تکمیلی آزمایش 5 مشخص شود.

باری، پیش گزارش این آزمایش شامل دو فایل زیپ است. یکی قبل از ددلاین اصلی، و دیگری اصلاحی و تکمیلی که می‌خواستیم موقع آپلود گزارش (17 آذر، با همین فایل) ارسال کنیم ولی فهمیدیم در سایت امکان ارسال هست، پس کنار قبلی اضافه کردیم.

این فایل صرفا گزارش کارها و چالش های آزمایشگاه و درباره پیاده سازی عملی مدار است:

1. آزمایش اصلی

ابتدا همین کدی که برای توصیف اصلی آزمایش پیاده سازی کردیم را روی FPGA تست کردیم. پس از کمی اشکال زدایی (مثلا عملکرد done کمی ایراد داشت)، نتیجه را به صورت درست مشاهده کردیم.

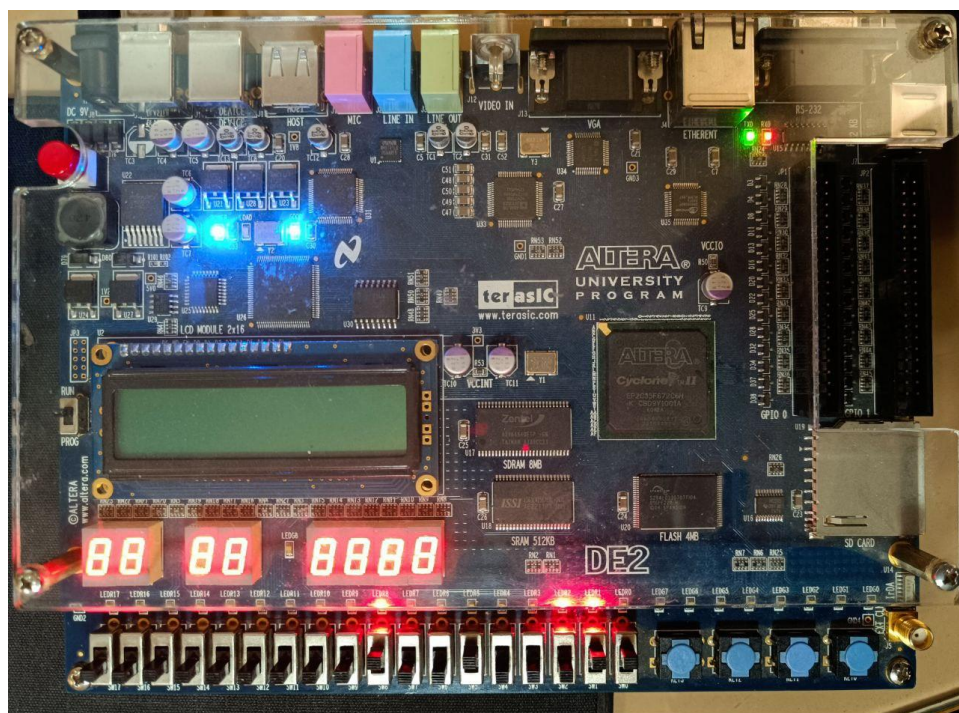
پین ها بدین صورت هستند که LEDR[0] ... LEDR[7] معادل با خروجی حاصل ضرب مدار هستند. LEDR[8] معادل با done است.

ورودی های بدین صورت هستند که sw3...sw0 معادل multiplicand (M) هستند. چهار بیت بعدی، یعنی sw4 ... sw7 معادل multiplier (Q) هستند. sw8 معادل reset و key1 معادل کلاک است.

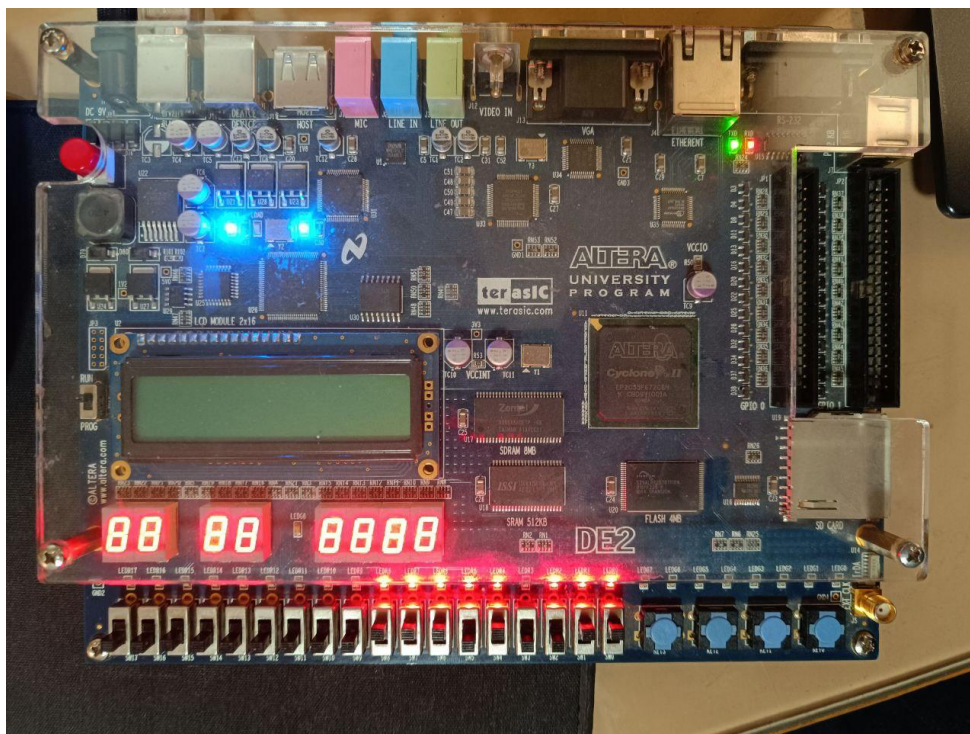
توجه شود برای راحتی تست مدار با همین ورودی های 4 بیتی را تست کردیم. مدار قابلیت تست اعداد با بیت های بیشتر را هم دارد.

خدا رو شکر همه چیز درست کار کرد. دو مثال را اینجا آوردیم.

- این ضرب دو عدد مثبت است. حاصل 3×2 به درستی 6 محاسبه شده است و done هم به درستی یک شده است:



- این هم ضرب دو عدد غیر هم علامت است. حاصل $3 \times (-3)$ به درستی 9- (1110111) بدست آمده است و done هم به درستی یک شده است:



2. اضافه کردن قابلیت شمردن تعداد کلاک ها و جمع ها و تفریق ها

در مرحله بعد قرار شد که تعداد پالس ساعت ها و همچنین تعداد جمع و تفریق ها محاسبه شود. پیاده سازی این مورد زیاد سخت نبود. کافی بود که data path را تغییر دهیم. در تمام کلاک ها clk_count یکی اضافه شود. مشابه برای add_count، وقتی به علاوه یک می شود که جمع رخ داده و sub_count وقتی بیشتر می شود که تفریق رخ داده است. انتظار داریم $clk_count = add_count + sub_count$.

این کار به این دلیل است که متوجه شویم آیا قابلیت چند شیفتر در یک کلاک و صرفه جویی در زمان به درستی اجرا می شود یا نه.

```
end else if (~done) begin
    operator <= operator >> shift_b;
    clk_count <= clk_count + 1;
    if (operation_select) begin
        result <= result + (Acc << shift_a);
        add_count <= add_count + 1;
    end else begin
        result <= result - (Acc << shift_a);
        sub_count <= sub_count + 1;
    end
end else if (done) begin
```

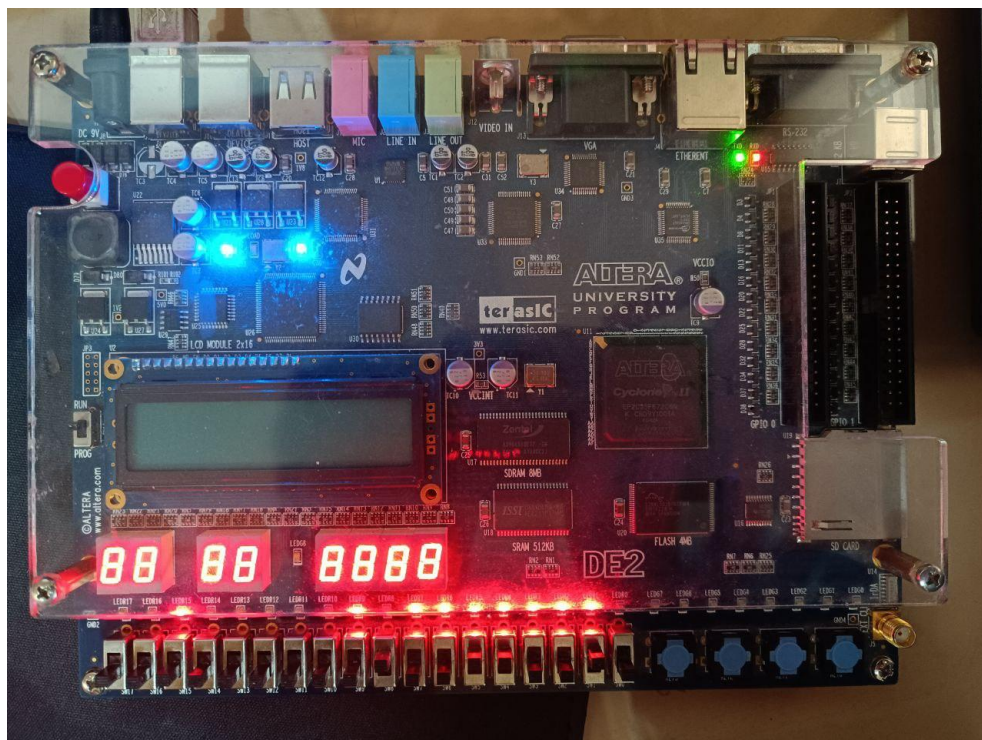
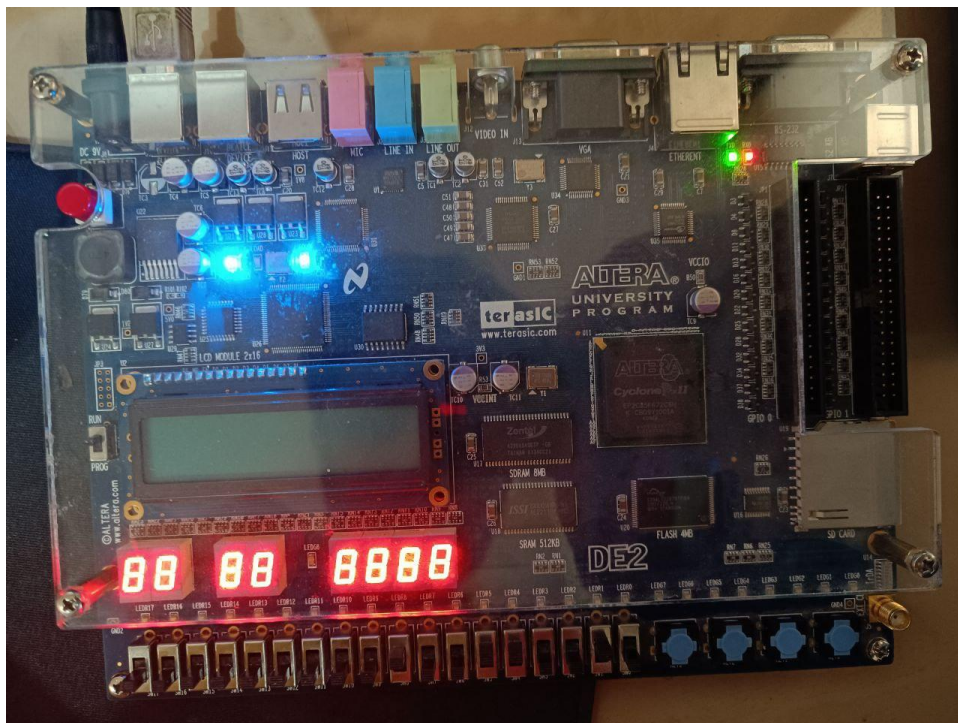
(این در کد نهایی که به پیوست گزارش ارسال می شود هم موجود است)

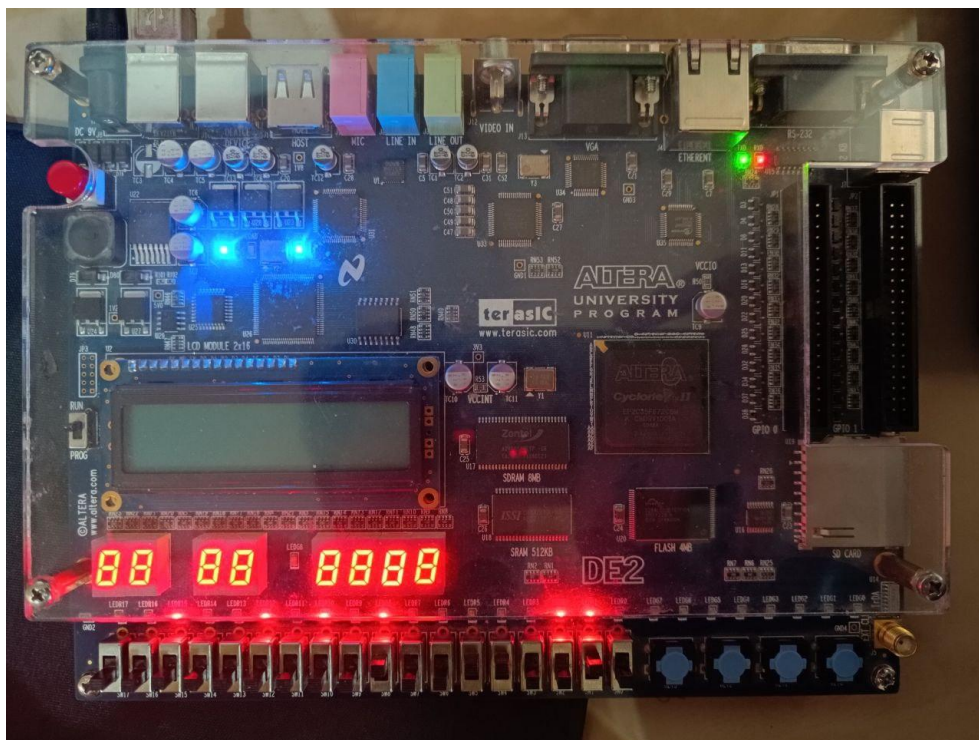
برای مشخص شدن خروجی، سه بیت sw11...sw9 مربوط به clk_count است، یعنی تعداد کل پالس ساعت ها برای محاسبه خروجی. سه بیت بعدی، یعنی sw14...sw12 برای add_count و سه بیت دیگر هم (sw17..sw15) برای sub_count هستند.

برای مثال در ضرب اعداد 2 و 3 با معادل باینری 0010 و 0011، $clk_count = 2$ میشود. زیرا در این مثال ما $Q = 3$ است (دقت شود چهار بیت sw7...sw4 مربوط به Q یا همان multiplier است). این به صورت 0011 است. وقتی یک صفر در سمت راست عدد می آید، عدد به صورت 00110 میشود. در اینجا محاسبه حاصل ضرب دو پالس ساعت طول می کشد، زیرا کافی ست یک 10 و سپس یک 01 بررسی شود و دو یک پشت سر هم (11) و دو صفر پشت سر هم (00) در همان کلاک بررسی میشوند تا به بیت مخالف برسیم.

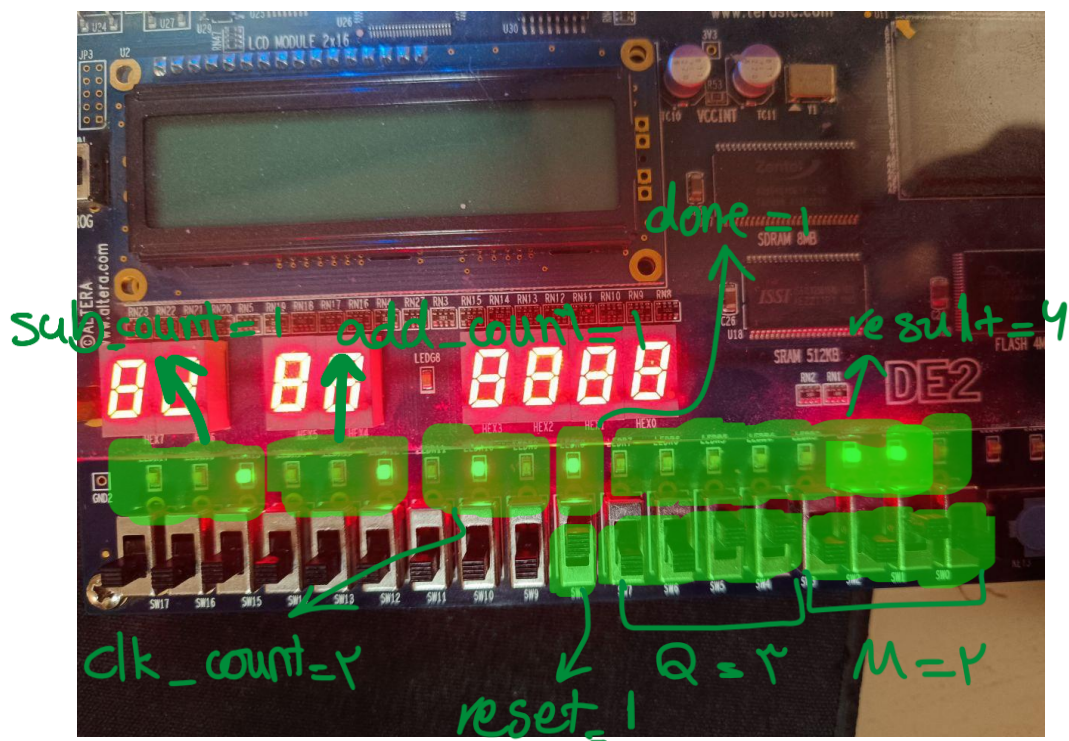
همانطور که انتظار می‌رفت، باید یک جمع به خاطر 01 و یک تفریق به خاطر 10 داشته باشیم و در نهایت `clk_count` = 2 شود. همه چیز مطابق انتظار است.

این دنباله 3 عکس بعدی روند اجرای این محاسبه را در دو کلاک نشان می‌دهد:





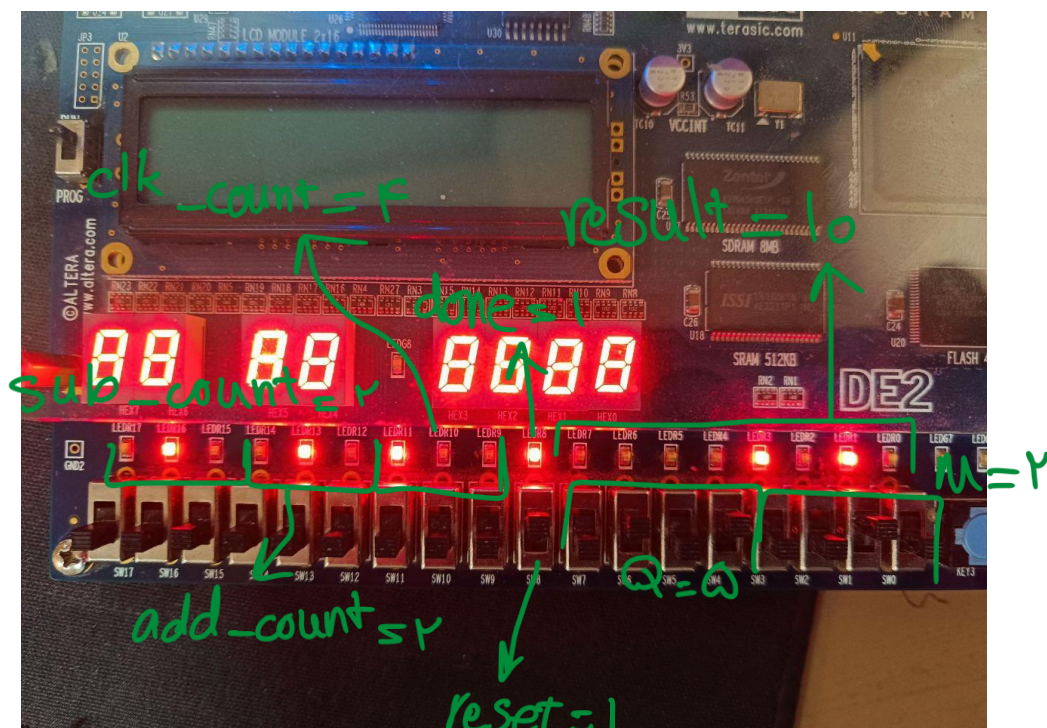
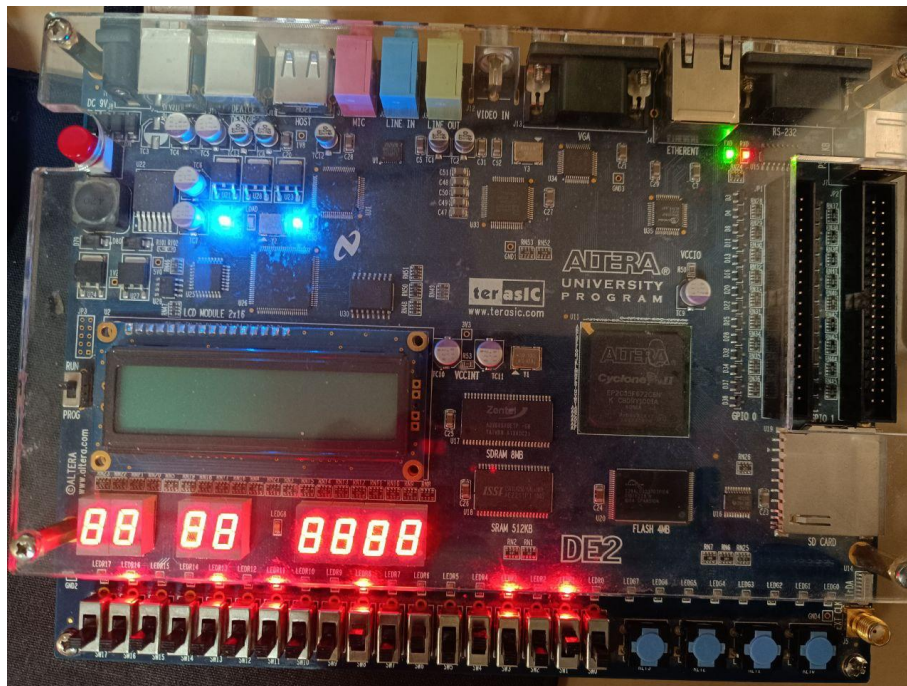
همانطور که مشخص است، به درستی خروجی های نشان داده می شوند. (عکس زیر هم بزرگ شده عکس بالا است)



بنده در عکس بالا هم به وضوح نتایج را نشان دادم

این هم یک مثال دیگر است که به درستی حاصل 2×5 نشان داده می‌شود. ($\text{multiplier} = 5$) پس عدد به صورت 0101 است. چیزی که باید بررسی شود 01010 است که با افزودن صفر در سمت راست 0101 بدست می‌آید. دو ترکیب 01 و دو ترکیب 10 داریم. پس $\text{add_count} = 2$ و $\text{sub_count} = 2$ میشود. clk_count هم به درستی 4 می‌شود.

این هم نتیجه نهایی این تست است:



3. اضافه کردن قابلیت بدست آوردن جمع چند حاصل ضرب

در گام بعد باید یک کار سخت انجام میدادیم. اینکه حاصل ضرب قبلی حفظ شود و پس از بدست آوردن حاصل ضرب و تغییر ورودی، حاصل ضرب دو ورودی بعدی با حاصل ضرب قبلی جمع شود (و این کار برای ورودی های بعدی هم ادامه داشته باشد).

برای این کار باید هم controller و هم datapath را تغییر میدادیم. (توجه شود که booth_mult.v هم اندکی تغییر میکند. مثلا ورودی ها و خروجی ها و سیم های متصل کننده ولی خب چیز قابل توضیحی ندارد. با این حال تمام مازول هایی که تغییر کردند، به پیوست ارسال شدند).

منطق controller چندان تغییر خاصی نمی‌کند. با یک trick عالی موفق شدیم با کمترین تغییر کد به هدف این task برسیم.

این trick این است که پس از اینکه نتیجه حاصل ضرب دو ورودی بدست آمد و با حاصل قبلی جمع شد، (یعنی is_added = 1 شد)، controller را reset کنیم (:). دقیقا منطق ریست واحد کنترلی را داریم. این کار را با تعریف pseudo_reset انجام دادیم:

```
assign pseudo_reset = ~(is_added || ~reset);
assign done = pseudo_reset ? (shift_a >= N) : 1'b0;
assign operation_select = start & Q[0];

assign shift_b = operation_select ? index0 : index1;
assign shift_a = done ? shift_a : (total_shift + shift_b);

always @(posedge clk, negedge pseudo_reset) begin
    if (~pseudo_reset) begin
        start <= 0;
        total_shift <= 0;
    end else begin
        start <= 1;
        total_shift <= total_shift + shift_b;
    end
end
```

حال برای datapath اما تغییرات بیشتر است. باید پس از اینکه حاصل ضرب قبلی بدست آمد و done برابر با یک شد، باید حاصل بدست آمده در این مرحله با حاصل جمع حاصل ضرب های قبلی جمع بزنیم. سپس is_added flag را یک کنیم به این معنا که جمع انجام شد.

سپس باید بلوک مربوط به `else if (is_added)` را بالاتر از `else if (~done)` اضافه کنیم منطق مشابه `reset` را در اینجا بنویسیم. دقت شود که عینا مشابه `reset` نیست چون `total` نباید صفر شود. (سایر سیگنال ها مثل `clk_count` را هم صفر کردیم تا صرفا نشان دهنده تعداد پالس در همین ضرب باشد)

وقتی `is_add` صفر میشود، در `controller` سیگنال `done` به درستی صفر میشود و سپس کد حاصل ضرب بعدی را محاسبه میکند.

(ما در لحظات آخر کلاس همانطور که متوجه شدید این `task` را انجام دادیم. ابتدا کدمان خطا می‌خورد ولی بعدا با سماجت بسیار (!) موفق شدیم کد درستی پیاده سازی کنیم. تنها مشکل این است که در این کد به نظر کمی منطق `controller` در `datapath` آورده شده است. خلاصه که تا جایی که فرصت شد تلاش کردیم کد خوبی پیاده سازی کنیم.))

این هم اصل منطق `datapath` است که تغییر کرد.

```
always @(posedge clk, negedge reset) begin
    if (~reset) begin
        operator <= Q;
        result <= 0;
        add_count <= 0;
        sub_count <= 0;
        clk_count <= 0;
        total <= 0;
        is_added <= 0;
    end
    else if (is_added) begin
        Acc <= {{N{M[N-1]}}, M};
        operator <= Q;
        result <= 0;
        add_count <= 0;
        sub_count <= 0;
        clk_count <= 0;
        is_added <= 0;
    end
    else if (~done) begin
        operator <= operator >> shift_b;
        clk_count <= clk_count + 1;
        if (operation_select) begin
            result <= result + (Acc << shift_a);
            add_count <= add_count + 1;
        end
        else begin
            result <= result - (Acc << shift_a);
            sub_count <= sub_count + 1;
        end
    end
    else if (done) begin
        total <= total + result;
        is_added <= 1;
    end
end
end
```

در نهایت یک تستی انجام دادیم ولی خب چون وقت تمام شده بود، متاسفانه صرفاً یک عکس از آن گرفتیم.

در اینجا ما ابتدا اعداد 2 و 2 را به عنوان ورودی دادیم و 4 را مشاهده کردیم. سپس ورودی ها را به 1 و 3 تغییر دادیم و

به درستی پاسخ 7 را مشاهده کردیم ($7 = 1*3 + 2*2$)

